Confluent Kafka 的最佳实践

NetApp Solutions

NetApp
April 07, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/netapp-solutions/data-analytics/confluent-kafka-introduction.html on April 07, 2022. Always check docs.netapp.com for the latest.

目录

Confluent Kafka 的最佳实践 . . . 1
TR-4912：《采用 NetApp 的 Confluent Kafka 分层存储最佳实践指南》 . . . 1
解决方案架构详细信息 . . . 2
技术概述 . . . 3
冲突验证 . . . 8
性能测试与可扩展性 . . . 11
融合 S3 连接器 . . . 13
融合的自平衡集群 . . . 22
最佳实践准则 . . . 22
规模估算 . . . 23
结论 . . . 26

Confluent Kafka 的最佳实践

TR-4912： 《采用 NetApp 的 Confluent Kafka 分层存储最佳实践指南》

Karthikeyan Nagalingam， Joseph Kandatilparambil， NetApp Rankesh Kumar， Confluent

Apache Kafka 是一个社区分布的事件流式平台，能够每天处理数万亿次的事件。Kafka 最初是一个消息队列，它基于分布式提交日志的抽象化。自从 2011 年由 LinkedIn 创建和开源以来， Kafka 已从消息队列发展为成熟的事件流式平台。Confluent 将 Apache Kafka 与 Confluent Platform 一起分发。Confluent 平台为 Kafka 提供了更多的社区和商业功能，旨在增强大规模生产中操作员和开发人员的流式体验。

本文档通过提供以下内容介绍在 NetApp 的对象存储产品上使用 Confluent 分层存储的最佳实践准则：

- 与 NetApp 对象存储— NetApp StorageGRID 相结合的验证
- 分层存储性能测试
- 在 NetApp 存储系统上使用 Confluent 的最佳实践准则

为什么选择 Confluent 分层存储?

对于许多应用程序来说， Confuent 已成为默认的实时流式平台，尤其是对于大数据，分析和流式工作负载。通过分层存储，用户可以将计算与 Confluent 平台中的存储分开。它可以提高数据存储的成本效益，使您能够存储几乎无限数量的数据并按需扩展工作负载，并使数据和租户重新平衡等管理任务变得更加轻松。与 S3 兼容的存储系统可以利用所有这些功能在一个位置实现数据的民主化，从而无需复杂的数据工程。有关为什么应为 Kafka 使用分层存储的详细信息，请查看 "本篇文章由 Confluent 提供"。

为什么要使用 NetApp StorageGRID 进行分层存储?

StorageGRID 是 NetApp 行业领先的对象存储平台。StorageGRID 是一款基于对象的软件定义存储解决方案，支持行业标准对象 API ，包括 Amazon Simple Storage Service （ S3 ） API 。StorageGRID 可大规模存储和管理非结构化数据，以提供安全，持久的对象存储。内容放置在合适的位置，合适的时间和合适的存储层上，从而优化工作流并降低全球分布式富媒体的成本。

StorageGRID 最大的竞争优势是其信息生命周期管理（ ILM ）策略引擎，该引擎支持策略驱动型数据生命周期管理。策略引擎可以使用元数据管理数据在其生命周期内的存储方式，以便在数据老化时对性能进行初始优化并自动优化成本和持久性。

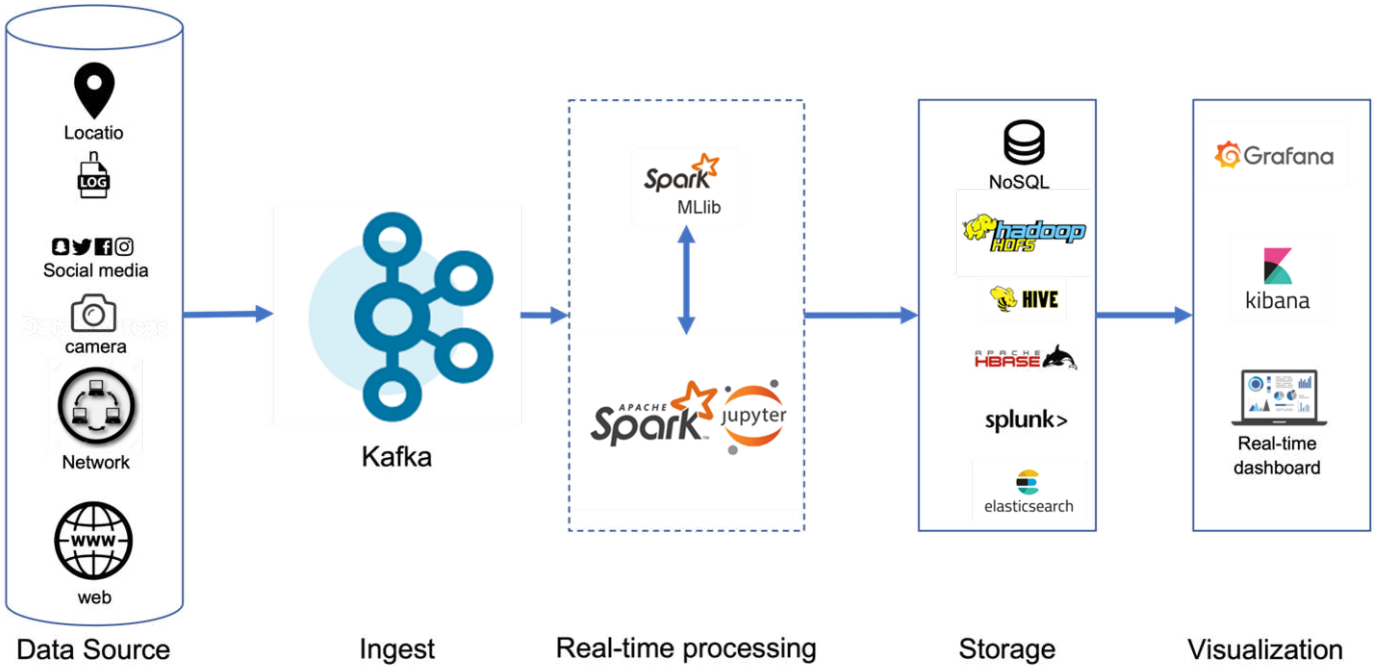
启用 Confluent 分层存储

分层存储的基本理念是，将数据存储任务与数据处理任务分开。通过这种分离，数据存储层和数据处理层独立扩展变得更加容易。

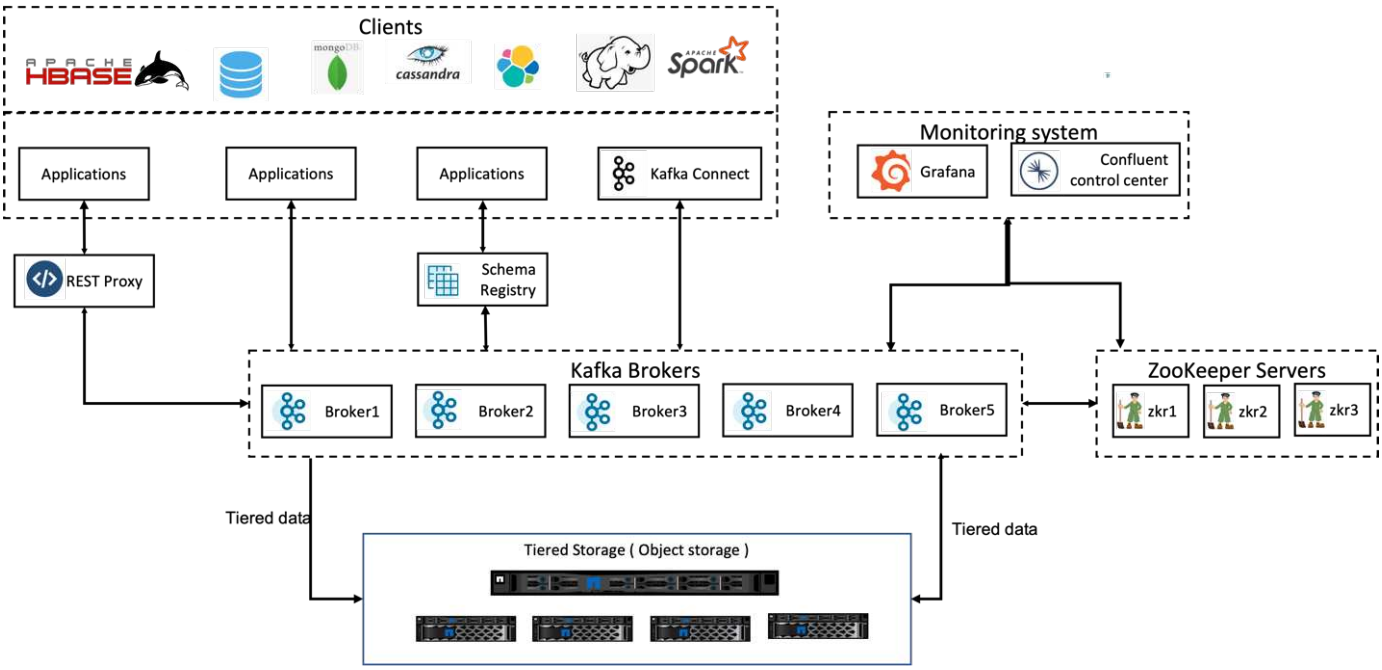
适用于 Confluent 的分层存储解决方案必须与两个因素相抗衡。首先， IT 必须解决或避免常见的对象存储一致性和可用性属性，例如列表操作不一致以及偶尔出现的对象不可用性。其次， IT 必须正确处理分层存储与 Kafka 的复制和容错模型之间的交互，包括 zombie 领导者继续分层偏移范围的可能性。NetApp 对象存储可提供一致的对象可用性和 HA 模式，从而使陈旧的存储可用于分层偏移范围。NetApp 对象存储可提供一致的对象可用性和 HA 模式，使陈旧的存储可用于分层偏移范围。

借助分层存储，您可以使用高性能平台在流式数据末尾附近进行低延迟读写，还可以使用 NetApp StorageGRID

等更便宜且可扩展的对象存储来进行高吞吐量历史读取。我们还提供了适用于采用 NetApp 存储控制器的 Spark 的技术解决方案，详细信息请参见此处。下图显示了 Kafka 如何融入实时分析管道。



下图展示了 NetApp StorageGRID 如何成为 Confluent Kafka 的对象存储层。



"接下来：解决方案架构详细信息。"

解决方案架构详细信息

"上一页：简介。"

本节介绍了用于 Confluent 验证的硬件和软件。此信息适用于使用 NetApp 存储部署 Confluent Platform 。下表

介绍了经过测试的解决方案架构和基本组件。

解决方案组件	详细信息
Confluent Kafka 6.2 版	• 三个 Zookepers • 五个代理服务器 • 五个工具服务器 • 一个 Grafana • 一个控制中心
Linux （ Ubuntu 18.04 ）	所有服务器
适用于分层存储的 NetApp StorageGRID	• StorageGRID 软件 • 1 个 SG1000 （负载平衡器） • 4 个 SGF6024 • 4 个 24 x 800 SSD • S3 协议 • 4 个 100GbE （代理和 StorageGRID 实例之间的网络连接）
15 台 Fujitsu PRIMERGY RX2540 服务器	每个均配备： * 2 个 CPU ，总共 16 个物理核心 * Intel Xeon * 256 GB 物理内存 * 100GbE 双端口

"接下来：技术概述"

技术概述

"先前版本：解决方案架构详细信息。"

本节介绍此解决方案中使用的技术。

NetApp StorageGRID

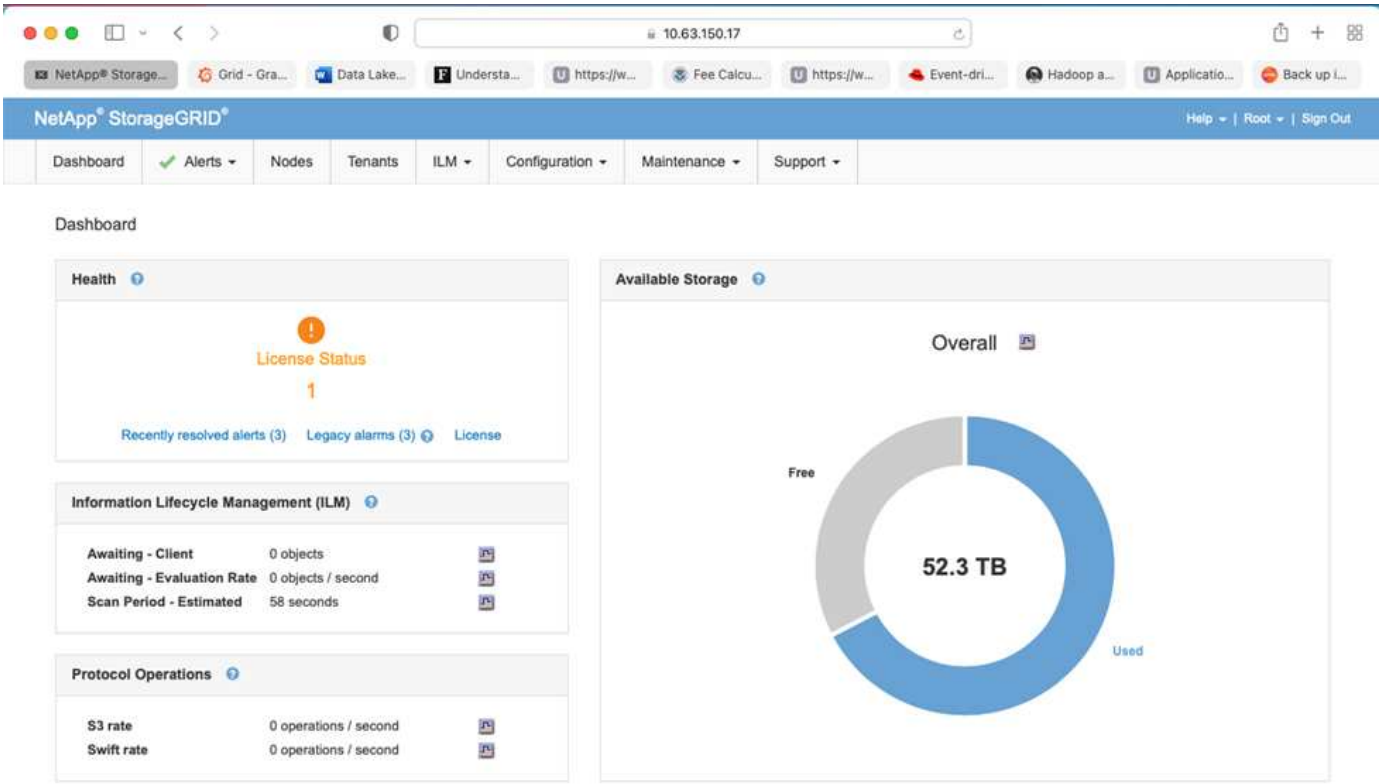
NetApp StorageGRID 是一款高性能，经济高效的对象存储平台。通过使用分层存储，存储在代理的本地存储或 SAN 存储中的 Confluent Kafka 上的大部分数据将卸载到远程对象存储。此配置可通过减少重新平衡，扩展或缩减集群或更换故障代理所需的时间和成本，显著改善运营。对象存储在管理对象存储层上的数据方面发挥着重要作用，因此选择合适的对象存储非常重要。

StorageGRID 采用基于节点的分布式网格架构，提供智能的策略驱动型全局数据管理。它通过其无处不在的全局对象命名空间以及复杂的数据管理功能，简化了对数 PB 的非结构化数据和数十亿个对象的管理。单次调用对象访问可扩展到各个站点，并简化高可用性架构，同时确保无论站点或基础架构是否中断，都能持续访问对象。

多租户支持在同一网格中安全地处理多个非结构化云和企业数据应用程序，从而提高 NetApp StorageGRID 的 ROI 并增加其用例。您可以使用元数据驱动型对象生命周期策略创建多个服务级别，以优化多个地理位置的持久性，保护，性能和位置。用户可以调整数据管理策略并监控和应用流量限制，以便在不断变化的 IT 环境中需求发生变化时无中断地与数据环境重新对齐。

使用 **Grid Manager** 进行简单管理

StorageGRID 网格管理器是一个基于浏览器的图形界面，可用于在一个管理平台中跨全球分布位置配置，管理和监控 StorageGRID 系统。



您可以使用 StorageGRID 网格管理器界面执行以下任务：

- 管理全局分布的 PB 级对象存储库，例如图像，视频和记录。
- 监控网格节点和服务以确保对象可用性。
- 使用信息生命周期管理（ ILM ）规则管理对象数据随时间的放置。这些规则用于控制在载入对象数据后该数据会发生什么情况，如何防止其丢失，对象数据的存储位置以及数据的存储时间。
- 监控系统内的事务，性能和操作。

信息生命周期管理策略

StorageGRID 具有灵活的数据管理策略，其中包括保留对象的副本以及使用 2+1 和 4+2 等 EC （纠删编码）方案来存储对象，具体取决于特定的性能和数据保护要求。随着工作负载和要求随时间的变化， ILM 策略也往往会随时间的变化而变化。修改 ILM 策略是一项核心功能，可使 StorageGRID 客户快速轻松地适应不断变化的环境。请检查 "ILM 策略" 和 "ILM 规则" 在 StorageGRID 中设置。

性能

StorageGRID 可通过添加更多存储节点来扩展性能，这些节点可以是 VM ，裸机或专门构建的设备，如 "SG5712 ， SG5760 ， SG6060 或 SGF6024"。在我们的测试中，使用 SGF6024 设备时，使用最小大小的三节点网格，超出了 Apache Kafka 的主要性能要求。随着客户通过更多代理扩展 Kafka 集群，他们可以添加更多存储节点以提高性能和容量。

负载平衡器和端点配置

StorageGRID 中的管理节点提供了网格管理器 UI （用户界面）和 REST API 端点，用于查看，配置和管理 StorageGRID 系统，并可通过审核日志来跟踪系统活动。为了为 Confluent Kafka 分层存储提供高可用性的 S3 端点，我们实施了 StorageGRID 负载平衡器，该平衡器作为一项服务在管理节点和网关节点上运行。此外，负载平衡器还管理本地流量并与 GSLB （全局服务器负载平衡）进行通信，以帮助进行灾难恢复。

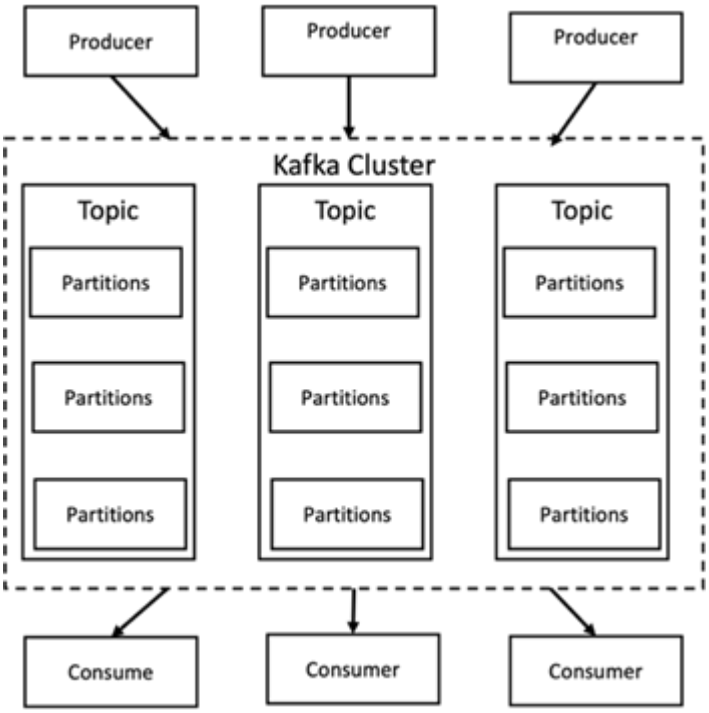
为了进一步增强端点配置， StorageGRID 提供了内置于管理节点中的流量分类策略，可用于监控工作负载流量，并对工作负载应用各种服务质量（ QoS ）限制。流量分类策略会应用于网关节点和管理节点的 StorageGRID 负载平衡器服务上的端点。这些策略有助于限制和监控流量。

StorageGRID 中的流量分类

StorageGRID 具有内置的 QoS 功能。流量分类策略有助于监控来自客户端应用程序的不同类型的 S3 流量。然后，您可以创建并应用策略，根据传入 / 输出带宽，读 / 写并发请求数或读 / 写请求率对此流量施加限制。

Apache Kafka

Apache Kafka 是一种使用 Java 和 Scala 编写的流处理的软件总线框架实施。它旨在提供一个统一的高吞吐量，低延迟平台来处理实时数据馈送。Kafka 可以连接到外部系统，以便通过 Kafka Connect 导出和导入数据，并提供 Kafka 流，这是一个 Java 流处理库。Kafka 使用基于 TCP 的二进制协议，该协议针对效率进行了优化，并依赖于 " 消息集 " 抽象，该抽象概念可将消息自然分组在一起，从而降低网络往返开销。这样可以实现更大的顺序磁盘操作，更大的网络数据包和连续的内存块，从而使 Kafka 能够将突发的随机消息写入流转变为线性写入。下图显示了 Apache Kafka 的基本数据流。



Kafka 存储来自任意数量的进程的关键价值消息，这些进程称为 " 生产者 " 。可以将数据分区为不同主题中的不同分区。在分区中，消息严格按其偏移量（消息在分区中的位置）排序，并编制索引并随时间戳一起存储。其他称为使用者的进程可以从分区读取消息。对于流处理， Kafka 提供了流 API ，可用于写入使用 Kafka 数据的 Java 应用程序，并将结果写入 Kafka 。Apache Kafka 还可与外部流处理系统配合使用，例如 Apache Apex ， Apache Flink ， Apache Spark ， Apache Storm 和 Apache NiFi 。

Kafka 在一个或多个服务器（称为代理）组成的集群上运行，所有主题的分区分布在各个集群节点上。此外，分

区会复制到多个代理。这种架构使 Kafka 能够以容错方式提供大量消息流，并使其能够取代一些传统消息传送系统，例如 Java 消息服务（ Java Message Service ， JMS ），高级消息队列协议（ Advanced Message Queuing Protocol ， AMQP ）等。自 0.11.0.0 版本以来， Kafka 提供了事务写入功能，可使用流 API 精确处理一次流。

Kafka 支持两种类型的主题：常规和压缩。可以为常规主题配置保留时间或空间限制。如果存在早于指定保留时间的记录，或者超出分区的绑定空间，则允许 Kafka 删除旧数据以释放存储空间。默认情况下，主题的保留时间为 7 天，但也可以无限期地存储数据。对于压缩主题，记录不会根据时间或空间限制过期。相反， Kafka 会将后续消息视为对具有相同密钥的旧消息的更新，并保证不会删除每个密钥的最新消息。用户可以通过使用特定密钥的空值编写所谓的 tombstone 消息来完全删除消息。

Kafka 中有五个主要 API：

- * 生成程序 API-* 允许应用程序发布记录流。
- * 使用者 API-* 允许应用程序订阅主题并处理记录流。
- * 连接器 API" 。 * 执行可重复使用的生产者和使用者 API ，以便将这些主题链接到现有应用程序。
- * 流 API 。 * 此 API 将输入流转换为输出并生成结果。
- * 管理 API-* 用于管理 Kafka 主题，代理和其他 Kafka 对象。

消费者和生产者 API 基于 Kafka 消息传送协议构建，并在 Java 中为 Kafka 消费者和生产者客户端提供了参考实施。底层消息传送协议是一种二进制协议，开发人员可以使用该协议以任何编程语言编写自己的使用者或生产者客户端。这样便可解除 Kafka 对 Java 虚拟机（ JVM ）生态系统的锁定。可用的非 Java 客户端列表会保存在 Apache Kafka wiki 中。

Apache Kafka 用例

Apache Kafka 在消息传送，网站活动跟踪，指标，日志聚合，流处理， 事件源和提交日志记录。

- Kafka 提高了吞吐量，内置分区，复制和容错功能，使其成为大规模消息处理应用程序的良好解决方案。
- Kafka 可以在跟踪管道中重建用户的活动（页面视图，搜索），并将其作为一组实时发布订阅源。
- Kafka 经常用于运行监控数据。其中包括汇总分布式应用程序的统计信息，以生成集中式运营数据源。
- 许多人使用 Kafka 代替日志聚合解决方案。日志聚合通常从服务器中收集物理日志文件，并将其置于中央位置（例如文件服务器或 HDFS ）进行处理。Kafka 可对文件详细信息进行抽象，并将日志或事件数据更清晰地抽象为一个消息流。这样可以降低延迟处理，并更轻松地支持多个数据源和分布式数据使用。
- Kafka 的许多用户会在由多个阶段组成的处理管道中处理数据，在这些阶段中，原始输入数据会从 Kafka 主题中使用，然后进行聚合，丰富或转换为新主题，以供进一步使用或进行后续处理。例如，用于推荐新闻文章的处理管道可能会从 rss 源中搜寻文章内容并将其发布到 " 文章 " 主题。进一步处理可能会使此内容规范化或进行重复数据删除，并将经过清理的文章内容发布到新主题中，最终处理阶段可能会尝试向用户推荐此内容。此类处理管道会根据各个主题创建实时数据流图形。
- 事件源化是一种应用程序设计模式，其状态更改将记录为按时间顺序排列的记录序列。Kafka 支持存储的非常大的日志数据，因此它是以这种模式构建的应用程序的理想后端。
- Kafka 可以用作分布式系统的一种外部提交日志。此日志有助于在节点之间复制数据，并充当故障节点恢复数据的重新同步机制。Kafka 中的日志缩减功能有助于支持此用例。

两者结合

Confluent Platform 是一款企业就绪平台，为 Kafka 提供了高级功能，旨在帮助加快应用程序开发和连接速度，

通过流处理实现转型，大规模简化企业运营并满足严格的架构要求。Confluent 由 Apache Kafka 的原始创建者构建，通过企业级功能扩展了 Kafka 的优势，同时消除了 Kafka 的管理或监控负担。如今，《财富》 100 强企业中有 80% 以上的企业都采用数据流技术，其中大多数企业都采用了流畅技术。

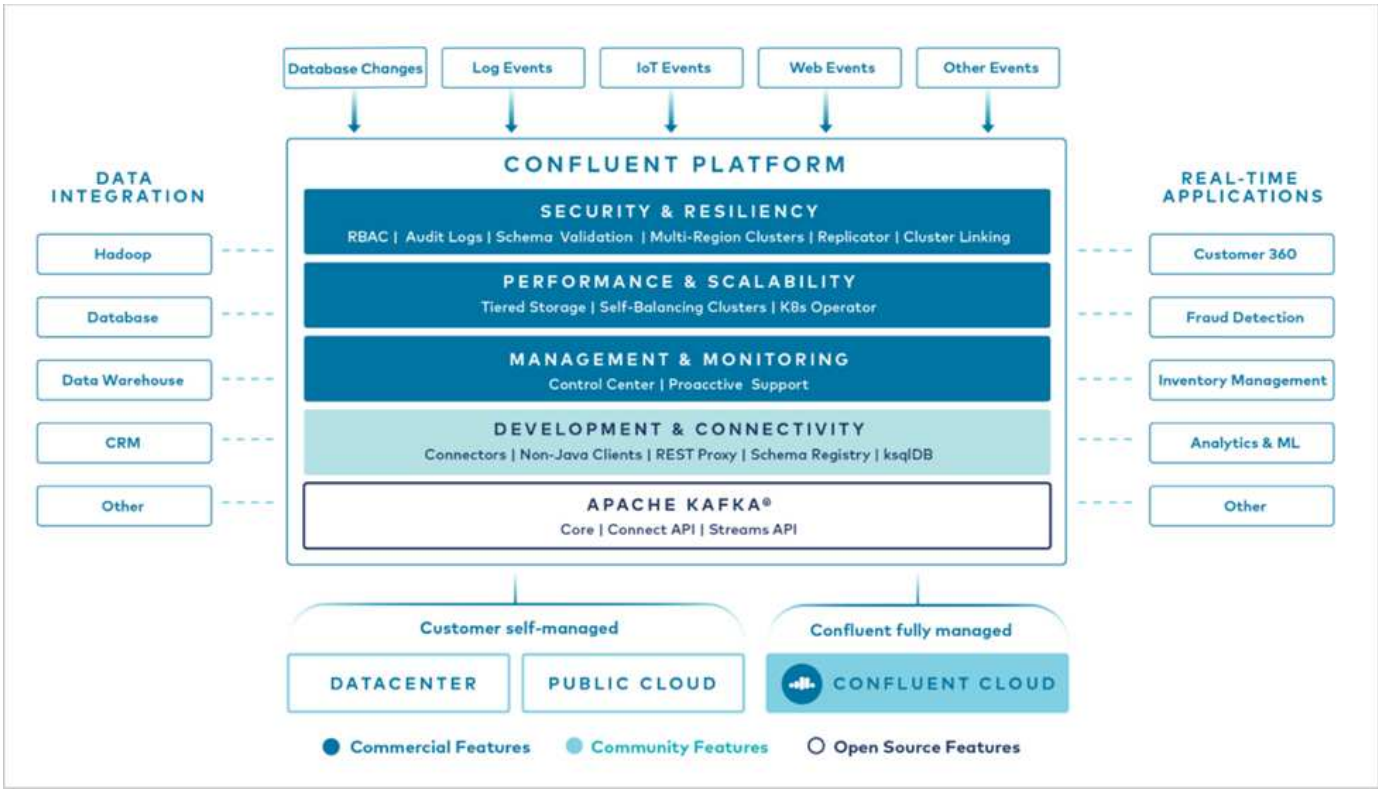
为什么选择 **Confluent** ？

通过将历史数据和实时数据集成到一个统一的中央真相来源中， Confluent 可以轻松构建一个全新的现代化事件驱动型应用程序类别，获得通用数据管道，并充分扩展性，性能和可靠性，释放出强大的新用例。

Confluent 的用途是什么?

借助整合平台，您可以专注于如何从数据中获得业务价值，而不是担心底层机制，例如如何在不同系统之间传输或集成数据。具体而言， Confluent Platform 可简化将数据源连接到 Kafka 的过程，构建流式应用程序，以及保护，监控和管理 Kafka 基础架构。如今， Consfluent Platform 已广泛用于各行各业的各种用例，从金融服务，全渠道零售和自动驾驶汽车到欺诈检测， 微服务和物联网。

下图显示了 Confluent Kafka 平台的组件。



Confluent 事件流技术概述

Confluent Platform 的核心是 "Apache Kafka"一种最受欢迎的开源分布式流式平台。Kafka 的主要功能如下：

- 发布并订阅记录流。
- 以容错方式存储记录流。
- 处理记录流。

即装即用的 Confluent 平台还包括架构注册表， REST 代理，总共 100 多个预构建的 Kafka 连接器和 ksqlDB 。

Confluent 平台企业功能概述

- * 流畅控制中心 * 。一种基于 GUI 的系统，用于管理和监控 Kafka 。您可以通过它轻松管理 Kafka Connect ，以及创建，编辑和管理与其他系统的连接。
- * Kubernetes 的 Confluent 。 * Kubernetes 的 Confluent 是 Kubernetes 的操作员。Kubernetes 操作员通过为特定平台应用程序提供独特的功能和要求，扩展了 Kubernetes 的业务流程功能。对于 Confluent Platform ，这包括大幅简化 Kubernetes 上 Kafka 的部署流程，并自动执行典型的基础架构生命周期任务。
- * 连接 Kafka 的流畅连接器。 * 连接器使用 Kafka Connect API 将 Kafka 连接到数据库，密钥值存储，搜索索引和文件系统等其他系统。Confluent Hub 提供可下载的连接器，用于最常用的数据源和数据池，包括这些连接器经过全面测试且受支持的版本以及 Confluent 平台。有关更多详细信息，请参见 "此处"。
- * 自平衡集群。 * 提供自动化负载平衡，故障检测和自我修复功能。它支持根据需要添加或停用代理，无需手动调整。
- * 流畅集群链接。 * 直接将集群连接在一起，并通过链路网桥将主题从一个集群镜像到另一个集群。集群链接可简化多数据中心，多集群和混合云部署的设置。
- * 流畅自动数据平衡器。 * 监控集群中的代理数量，分区大小，分区数量和导数。它允许您在集群中移动数据以创建均匀的工作负载，同时限制重新平衡流量，以便在重新平衡的同时最大限度地减少对生产工作负载的影响。
- * 流畅复制器。 * 使在多个数据中心维护多个 Kafka 集群变得比以往任何时候都更轻松。
- * 分层存储。 * 提供了使用您喜欢的云提供商存储大量 Kafka 数据的选项，从而减轻了运营负担并降低了成本。借助分层存储，您只能在需要更多计算资源时，才可以将数据保存在经济高效的对象存储和扩展代理上。
- * 流畅的 jms 客户端。 * 流畅平台包括适用于 Kafka 的与 jms 兼容的客户端。此 Kafka 客户端使用 Kafka 代理作为后端，实施了 Jms 1.1 标准 API 。如果旧版应用程序使用的是 jms ，并且您希望将现有的 jms 消息代理替换为 Kafka ，则此功能非常有用。
- * 流畅的 MQT 代理。 * 提供了一种从 MQT 设备和网关直接向 Kafka 发布数据的方法，而无需在中间使用 MQT 代理。
- * 流畅安全插件。 * 流畅安全插件用于为各种流畅平台工具和产品添加安全功能。目前，可以为 Confluent REST 代理提供一个插件，用于对传入请求进行身份验证，并将经过身份验证的主体传播到 Kafka 请求。这样， Confluent REST 代理客户端便可利用 Kafka 代理的多租户安全功能。

"下一步：验证冲突。"

冲突验证

"上一页：技术概述。"

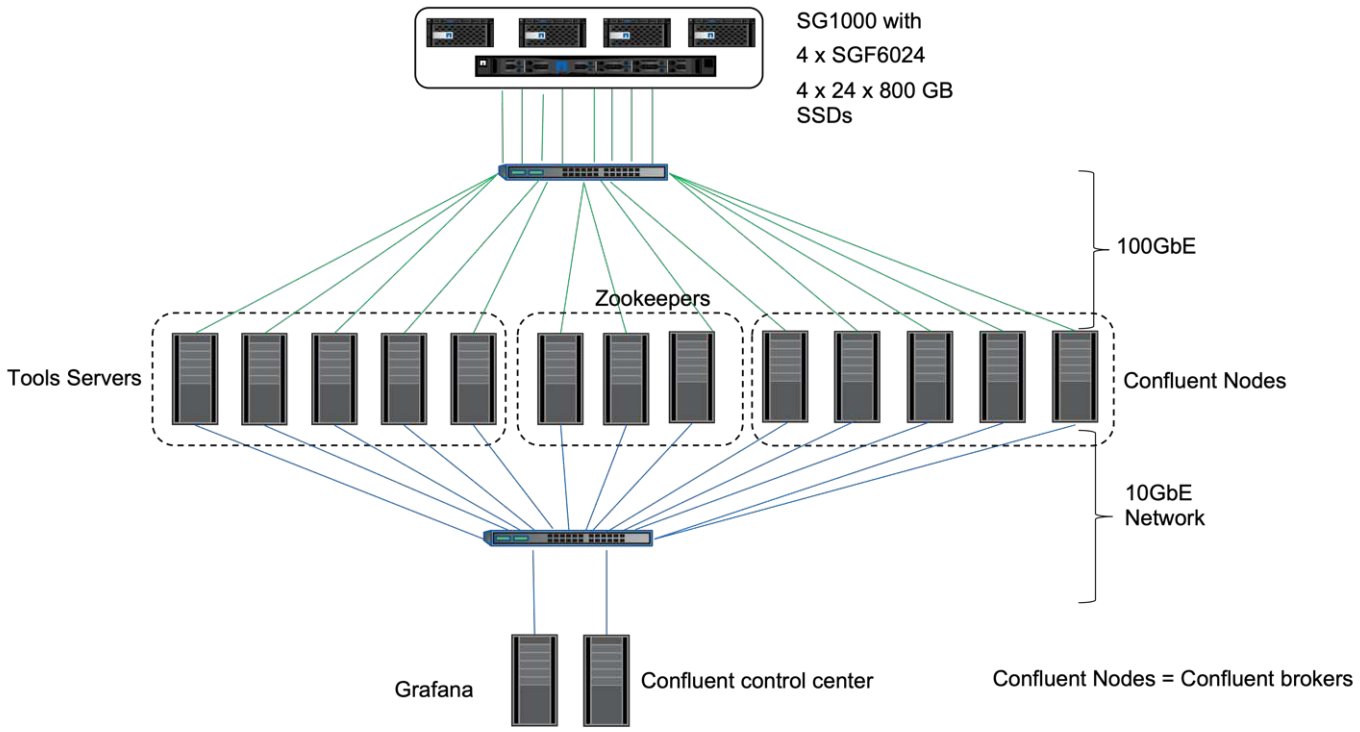
我们使用 NetApp StorageGRID 中的 Confluent Platform 6.2 分层存储执行了验证。NetApp 和 Confluent 团队共同执行了此验证，并运行了验证所需的测试用例。

Confuent Platform 设置

我们使用以下设置进行验证。

为了进行验证，我们使用了三个 Zookeepers ，五个代理，五个测试脚本执行服务器，命名为 Tools 服务器，该服务器具有 256 GB RAM 和 16 个 CPU 。对于 NetApp 存储，我们使用的是带有四个 SGF6024 的 SG1000 负载平衡器的 StorageGRID 。存储和代理通过 100GbE 连接进行连接。

下图显示了用于 Confluent 验证的配置的网络拓扑。



工具服务器充当向 Confluent 节点发送请求的应用程序客户端。

融合的分层存储配置

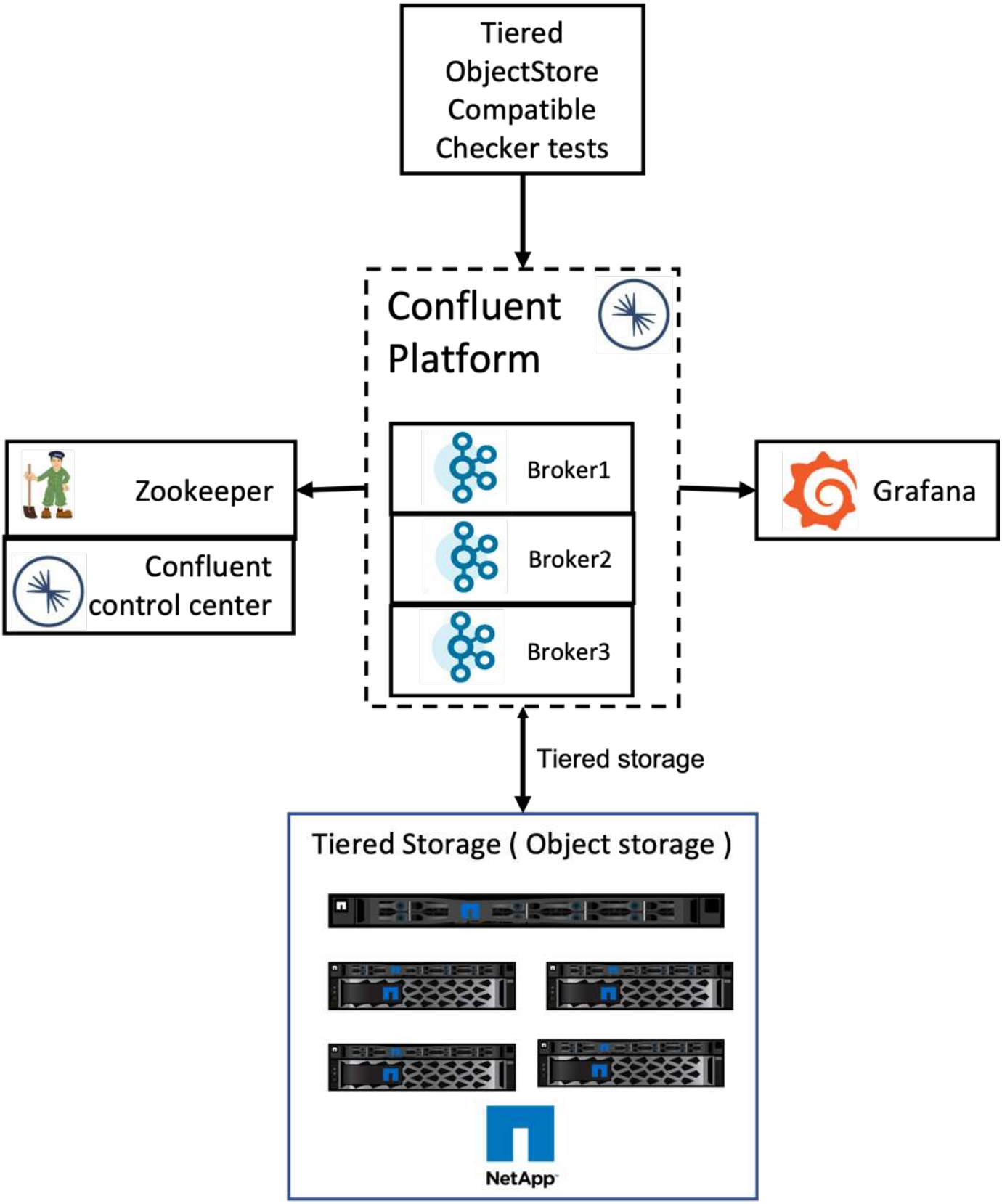
分层存储配置在 Kafka 中需要以下参数：

```
Confluent.tier.archiver.num.threads=16
confluent.tier.fetcher.num.threads=32
confluent.tier.enable=true
confluent.tier.feature=true
confluent.tier.backend=S3
confluent.tier.s3.bucket=kafkasgdbucket1-2
confluent.tier.s3.region=us-west-2
confluent.tier.s3.cred.file.path=/data/kafka/.ssh/credentials
confluent.tier.s3.aws.endpoint.override=http://kafkasgd.rtpppe.netapp.com:
10444/
confluent.tier.s3.force.path.style.access=true
```

为了进行验证，我们将 StorageGRID 与 HTTP 协议结合使用，但 HTTPS 也有效。访问密钥和机密密钥存储在 `confuent.tier.s3.cred.file.path` 参数中提供的文件名中。

NetApp 对象存储— StorageGRID

我们在 StorageGRID 中配置了单站点配置以进行分层。



验证测试

我们已完成以下五个测试案例以进行验证。这些测试将在 Trogdor 框架上执行。前两项是功能测试，其余三项是性能测试。

对象存储正确性测试

此测试将根据分层存储的需求确定对象存储 API 上的所有基本操作（例如 GET ， PUT 或 DELETE ）是否运行良好。这是一项基本测试，每个对象存储服务都应在以下测试之前通过。这是一项自信的测试，无论通过还是失败。

分层功能正确性测试

此测试可确定端到端分层存储功能是否运行良好，并通过或失败的自信测试。此测试将创建一个测试主题，默认情况下，此主题会配置为启用分层并大幅减小热设置大小。它会为新创建的测试主题生成一个事件流，并等待代理将这些分段归档到对象存储，然后使用事件流并验证已使用的流是否与生成的流匹配。生成给事件流的消息数量是可配置的，这样用户可以根据测试需求生成足够大的工作负载。减小的热集大小可确保活动分段之外的使用者提取仅从对象存储提供；这有助于测试对象存储的读取是否正确。我们执行此测试时，无论是否注入了对象存储故障。我们通过在 StorageGRID 中的一个节点中停止服务管理器服务并验证端到端功能是否适用于对象存储来模拟节点故障。

层提取基准测试

此测试验证了分层对象存储的读取性能，并检查了基准测试生成的区块在负载过重时的范围提取读取请求。在此基准测试中， Confluent 开发了自定义客户端来处理层提取请求。

生产 - 使用工作负载基准测试

此测试会通过归档区块在对象存储上间接生成写入工作负载。读取工作负载（区块读取）是在使用者组提取区块时从对象存储生成的。此工作负载由测试脚本生成。此测试检查了并行线程中对象存储上的读写性能。与分层功能正确性测试一样，我们测试了是否存在对象存储故障注入。

保留工作负载基准测试

此测试检查了在主题保留工作负载繁重的情况下对象存储的删除性能。保留工作负载是使用测试脚本生成的，该脚本会与测试主题并行生成许多消息。本测试主题使用主动式基于大小和基于时间的保留设置进行配置，此设置会导致从对象存储中持续清除事件流。然后，这些区块会归档。这导致代理在对象存储中删除了大量内容，并收集了对象存储删除操作的性能。

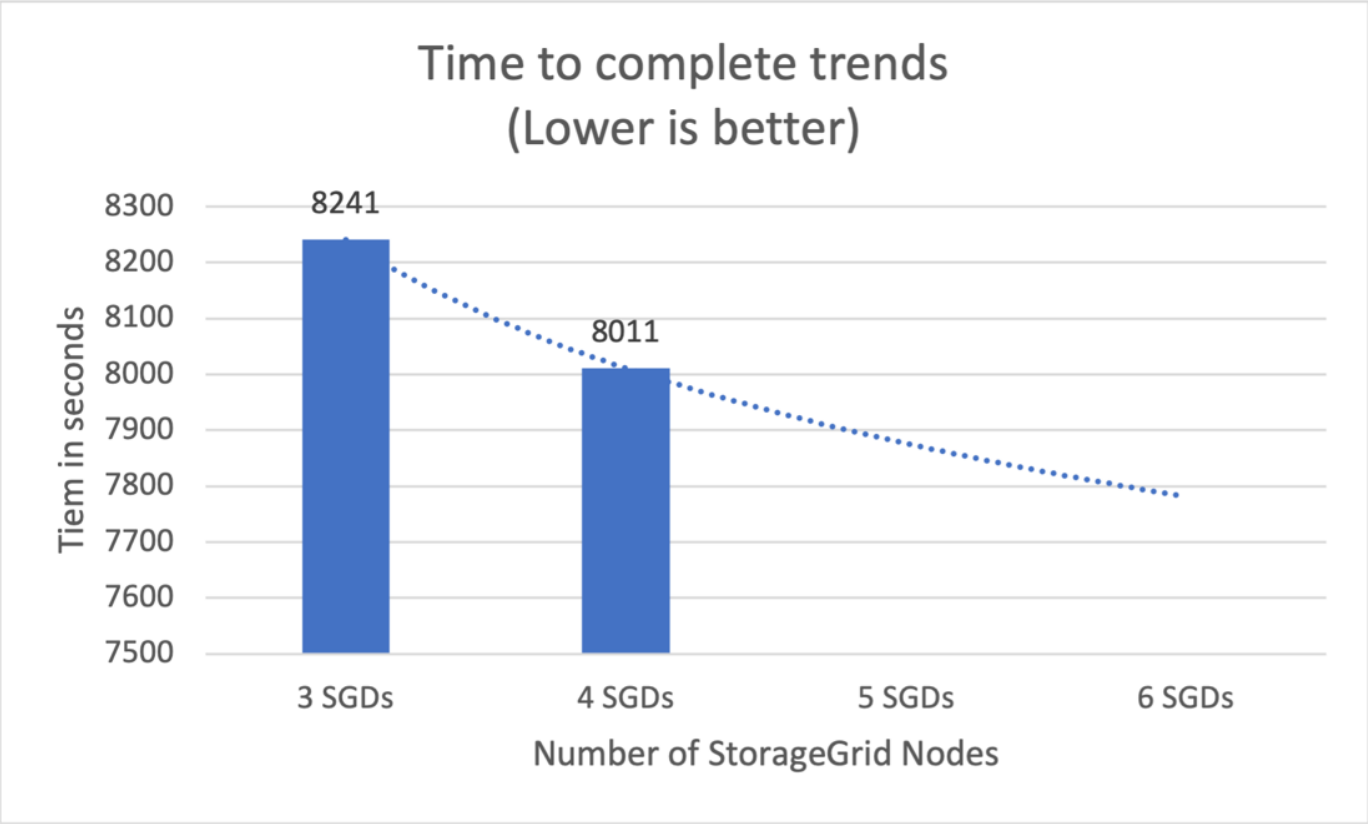
"接下来：性能测试和可扩展性。"

性能测试与可扩展性

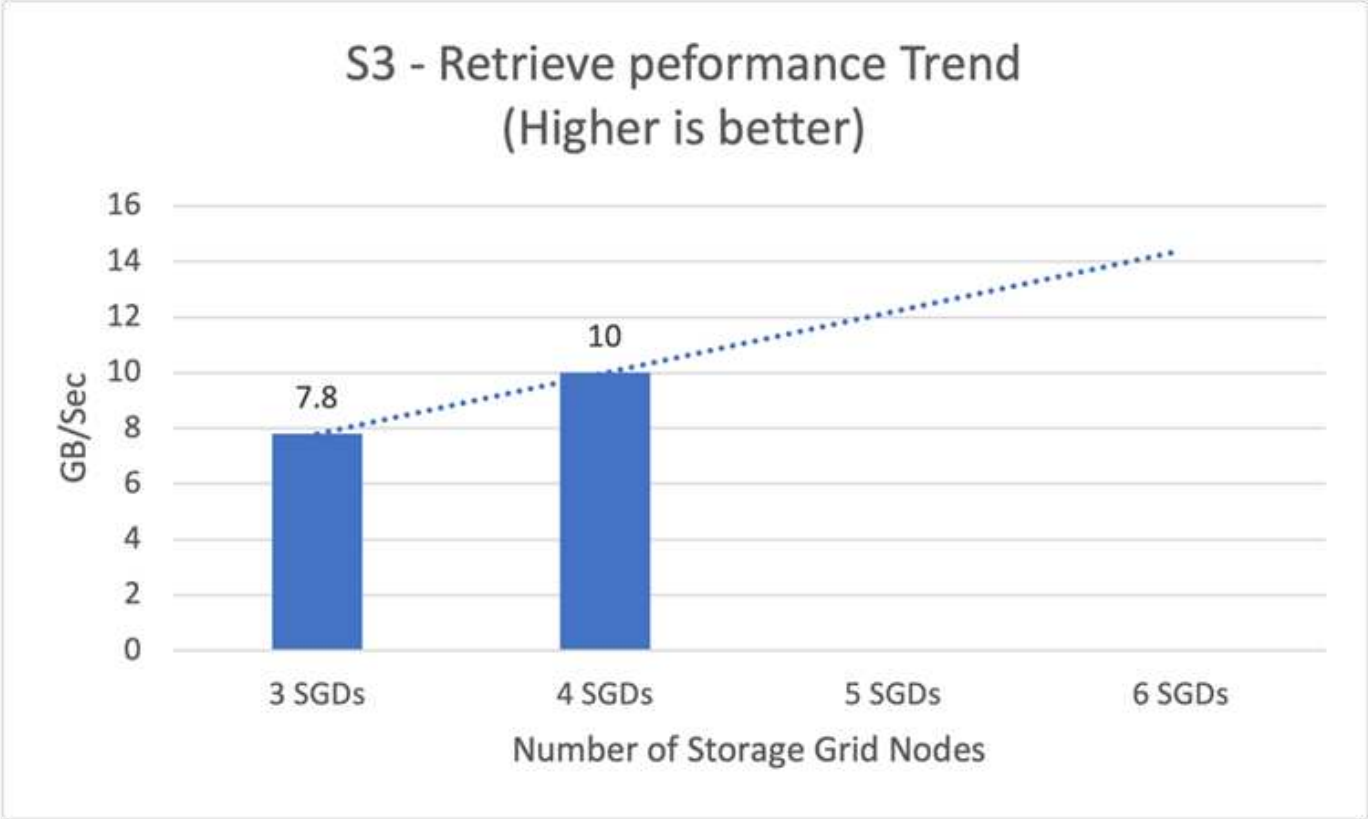
"Previous： Confuent verification 。"

我们使用 NetApp StorageGRID 设置对生产者和使用者工作负载执行了三到四个节点的分层存储测试。根据我们的测试，完成时间和性能结果与 StorageGRID 节点数成正比。StorageGRID 设置至少需要三个节点。

- 当存储节点数量增加时，完成生产和消费模式操作所需的时间呈线性下降趋势。



- 根据 StorageGRID 节点的数量， S3 检索操作的性能呈线性增长。StorageGRID 最多支持 200 个 StorgeGRID 节点。



"接下来：将 S3 连接器连接在一起。"

融合 S3 连接器

"上一篇：性能测试与可扩展性。"

Amazon S3 Sink Connector 以 Avro ， JSON 或字节格式将数据从 Apache Kafka 主题导出到 S3 对象。Amazon S3 Sink Connector 会定期轮询 Kafka 中的数据，然后将其上传到 S3 。分区程序用于将每个 Kafka 分区的数据拆分为多个区块。每个数据区块都表示为 S3 对象。密钥名称会对主题， Kafka 分区以及此数据块的起始偏移进行编码。

在此设置中，我们将向您展示如何使用 Kafka S3 接收器连接器直接从 Kafka 读取和写入对象存储中的主题。在此测试中，我们使用了独立的 Consfluent 集群，但此设置适用于分布式集群。

1. 从 Confluent 网站下载 Confluent Kafka 。
2. 将软件包解压缩到服务器上的文件夹。
3. 导出两个变量。

```
Export CONFLUENT_HOME=/data/confluent/confluent-6.2.0
export PATH=$PATH:/data/confluent/confluent-6.2.0/bin
```

4. 对于独立的 Confluent Kafka 设置，集群会在 ` /tmp` 中创建一个临时根文件夹。它还会创建 Zookeeper ， Kafka ，模式注册表， connect ， ksql-server ， 和控制中心文件夹，并从 ` $confuent_home` 复制其各自的配置文件。请参见以下示例：

```
root@stlrx2540m1-108:~# ls -ltr /tmp/confluent.406980/
total 28
drwxr-xr-x 4 root root 4096 Oct 29 19:01 zookeeper
drwxr-xr-x 4 root root 4096 Oct 29 19:37 kafka
drwxr-xr-x 4 root root 4096 Oct 29 19:40 schema-registry
drwxr-xr-x 4 root root 4096 Oct 29 19:45 kafka-rest
drwxr-xr-x 4 root root 4096 Oct 29 19:47 connect
drwxr-xr-x 4 root root 4096 Oct 29 19:48 ksql-server
drwxr-xr-x 4 root root 4096 Oct 29 19:53 control-center
root@stlrx2540m1-108:~#
```

5. 配置 Zookeeper 。如果使用默认参数，则无需更改任何内容。

```
root@stlrx2540m1-108:~# cat
/tmp/confluent.406980/zookeeper/zookeeper.properties  | grep -iv ^#
dataDir=/tmp/confluent.406980/zookeeper/data
clientPort=2181
maxClientCnxns=0
admin.enableServer=false
tickTime=2000
initLimit=5
syncLimit=2
server.179=controlcenter:2888:3888
root@stlrx2540m1-108:~#
```

在上述配置中，我们更新了 s服务器。xxx 属性。默认情况下，您需要三个 zookepers 来选择 Kafka 领导者。

6. 我们在 ` /tmp/confuent.406980/zookeeper /data` 中创建了一个 myid 文件，其唯一 ID 为：

```
root@stlrx2540m1-108:~# cat /tmp/confluent.406980/zookeeper/data/myid
179
root@stlrx2540m1-108:~#
```

我们使用 myid 文件的最后一个 IP 地址数。我们使用了 Kafka ， connect ， control-center ， Kafka ， Kafka-REST ， ksql-server 和模式注册表配置。

7. 启动 Kafka 服务。

```
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# confluent
local services  start
The local commands are intended for a single-node development
environment only,
NOT for production usage.

Using CONFLUENT_CURRENT: /tmp/confluent.406980
ZooKeeper is [UP]
Kafka is [UP]
Schema Registry is [UP]
Kafka REST is [UP]
Connect is [UP]
ksqlDB Server is [UP]
Control Center is [UP]
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

每个配置都有一个日志文件夹，可帮助您解决问题。在某些情况下，服务需要较长时间才能启动。确保所有服务均已启动且正在运行。

8. 使用 `Confluent-hub` 安装 Kafka 连接。

```
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# ./confluent-
hub install confluentinc/kafka-connect-s3:latest
The component can be installed in any of the following Confluent
Platform installations:
  1. /data/confluent/confluent-6.2.0 (based on $CONFLUENT_HOME)
  2. /data/confluent/confluent-6.2.0 (where this tool is installed)
Choose one of these to continue the installation (1-2): 1
Do you want to install this into /data/confluent/confluent-
6.2.0/share/confluent-hub-components? (yN) y

Component's license:
Confluent Community License
http://www.confluent.io/confluent-community-license
I agree to the software license agreement (yN) y
Downloading component Kafka Connect S3 10.0.3, provided by Confluent,
Inc. from Confluent Hub and installing into /data/confluent/confluent-
6.2.0/share/confluent-hub-components
Do you want to uninstall existing version 10.0.3? (yN) y
Detected Worker's configs:
  1. Standard: /data/confluent/confluent-6.2.0/etc/kafka/connect-
distributed.properties
  2. Standard: /data/confluent/confluent-6.2.0/etc/kafka/connect-
standalone.properties
  3. Standard: /data/confluent/confluent-6.2.0/etc/schema-
registry/connect-avro-distributed.properties
  4. Standard: /data/confluent/confluent-6.2.0/etc/schema-
registry/connect-avro-standalone.properties
  5. Based on CONFLUENT_CURRENT:
/tmp/confluent.406980/connect/connect.properties
  6. Used by Connect process with PID 15904:
/tmp/confluent.406980/connect/connect.properties
Do you want to update all detected configs? (yN) y
Adding installation directory to plugin path in the following files:
  /data/confluent/confluent-6.2.0/etc/kafka/connect-
distributed.properties
  /data/confluent/confluent-6.2.0/etc/kafka/connect-
standalone.properties
  /data/confluent/confluent-6.2.0/etc/schema-registry/connect-avro-
distributed.properties
  /data/confluent/confluent-6.2.0/etc/schema-registry/connect-avro-
standalone.properties
  /tmp/confluent.406980/connect/connect.properties
  /tmp/confluent.406980/connect/connect.properties
```

```
Completed
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

您也可以使用 `Confluent-hub install conflientint/Kafka-connect-S3 ： 10.0.3` 来安装特定版本。

9. 默认情况下， `confuentine-Kafka-connect-S3` 安装在 ` /data/confuent/confuent-6.2.0/share/confuent-hub-components/confuentine-Kafka-connect-S3` 中。
10. 使用新的 `Confluentine-Kafka-connect-S3` 更新插件路径。

```
root@stlrx2540m1-108:~# cat /data/confluent/confluent-
6.2.0/etc/kafka/connect-distributed.properties | grep plugin.path
#
plugin.path=/usr/local/share/java,/usr/local/share/kafka/plugins,/opt/co
nnectors,
plugin.path=/usr/share/java,/data/zookeeper/confluent/confluent-
6.2.0/share/confluent-hub-components,/data/confluent/confluent-
6.2.0/share/confluent-hub-components,/data/confluent/confluent-
6.2.0/share/confluent-hub-components/confluentinc-kafka-connect-s3
root@stlrx2540m1-108:~#
```

11. 停止并重新启动 Consfluent 服务。

```
confluent local services  stop
confluent local services  start
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# confluent
local services  status
The local commands are intended for a single-node development
environment only,
NOT for production usage.

Using CONFLUENT_CURRENT: /tmp/confluent.406980
Connect is [UP]
Control Center is [UP]
Kafka is [UP]
Kafka REST is [UP]
ksqlDB Server is [UP]
Schema Registry is [UP]
ZooKeeper is [UP]
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

12. 在 ` /root/.AWS/credentials` 文件中配置访问 ID 和机密密钥。

```
root@stlrx2540m1-108:~# cat /root/.aws/credentials
[default]
aws_access_key_id = xxxxxxxxxxxx
aws_secret_access_key = xxxxxxxxxxxxxxxxxxxxxxxxx
root@stlrx2540m1-108:~#
```

13. 验证存储分段是否可访问。

```
root@stlrx2540m4-01:~# aws s3 -endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 ls kafkasgdbucket1-2
2021-10-29 21:04:18       1388 1
2021-10-29 21:04:20       1388 2
2021-10-29 21:04:22       1388 3
root@stlrx2540m4-01:~#
```

14. 为 S3 和存储分段配置 S3-sink 属性文件。

```
root@stlrx2540m1-108:~# cat /data/confluent/confluent-
6.2.0/share/confluent-hub-components/confluentinc-kafka-connect-
s3/etc/quickstart-s3.properties | grep -v ^#
name=s3-sink
connector.class=io.confluent.connect.s3.S3SinkConnector
tasks.max=1
topics=s3_testtopic
s3.region=us-west-2
s3.bucket.name=kafkasgdbucket1-2
store.url=http://kafkasgd.rtpppe.netapp.com:10444/
s3.part.size=5242880
flush.size=3
storage.class=io.confluent.connect.s3.storage.S3Storage
format.class=io.confluent.connect.s3.format.avro.AvroFormat
partitioner.class=io.confluent.connect.storage.partitioner.DefaultPartit
ioner
schema.compatibility=NONE
root@stlrx2540m1-108:~#
```

15. 将一些记录导入到 S3 存储分段中。

```
kafka-avro-console-producer --broker-list localhost:9092 --topic
s3_topic \
--property
value.schema='{"type":"record","name":"myrecord","fields":[{"name":"f1",
"type":"string"}]}'
{"f1": "value1"}
{"f1": "value2"}
{"f1": "value3"}
{"f1": "value4"}
{"f1": "value5"}
{"f1": "value6"}
{"f1": "value7"}
{"f1": "value8"}
{"f1": "value9"}
```

16. 加载 S3-sink 连接器。

```
root@stlrx2540m1-108:~# confluent local services connect connector load
s3-sink  --config /data/confluent/confluent-6.2.0/share/confluent-hub-
components/confluentinc-kafka-connect-s3/etc/quickstart-s3.properties
The local commands are intended for a single-node development
environment only,
NOT for production usage.
https://docs.confluent.io/current/cli/index.html
{
  "name": "s3-sink",
  "config": {
    "connector.class": "io.confluent.connect.s3.S3SinkConnector",
    "flush.size": "3",
    "format.class": "io.confluent.connect.s3.format.avro.AvroFormat",
    "partitioner.class":
"io.confluent.connect.storage.partitioner.DefaultPartitioner",
    "s3.bucket.name": "kafkasgdbucket1-2",
    "s3.part.size": "5242880",
    "s3.region": "us-west-2",
    "schema.compatibility": "NONE",
    "storage.class": "io.confluent.connect.s3.storage.S3Storage",
    "store.url": "http://kafkasgd.rtpppe.netapp.com:10444/",
    "tasks.max": "1",
    "topics": "s3_testtopic",
    "name": "s3-sink"
  },
  "tasks": [],
  "type": "sink"
}
root@stlrx2540m1-108:~#
```

17. 检查 S3-sink 状态。

```
root@stlrx2540m1-108:~# confluent local services connect connector
status s3-sink
The local commands are intended for a single-node development
environment only,
NOT for production usage.
https://docs.confluent.io/current/cli/index.html
{
  "name": "s3-sink",
  "connector": {
    "state": "RUNNING",
    "worker_id": "10.63.150.185:8083"
  },
  "tasks": [
    {
      "id": 0,
      "state": "RUNNING",
      "worker_id": "10.63.150.185:8083"
    }
  ],
  "type": "sink"
}
root@stlrx2540m1-108:~#
```

18. 检查日志以确保 S3-sink 已准备好接受主题。

```
root@stlrx2540m1-108:~# confluent local services connect log
```

19. 查看 Kafka 中的主题。

```
kafka-topics --list --bootstrap-server localhost:9092
…
connect-configs
connect-offsets
connect-statuses
default_ksql_processing_log
s3_testtopic
s3_topic
s3_topic_new
root@stlrx2540m1-108:~#
```

20. 检查 S3 存储分段中的对象。

```
root@stlrx2540m1-108:~# aws s3 --endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 ls --recursive kafkasgdbucket1-
2/topics/
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000003.avro
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000006.avro
2021-10-29 21:24:08        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000009.avro
2021-10-29 21:24:08        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000012.avro
2021-10-29 21:24:09        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000015.avro
root@stlrx2540m1-108:~#
```

21. 要验证内容，请运行以下命令将每个文件从 S3 复制到本地文件系统：

```
root@stlrx2540m1-108:~# aws s3 --endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 cp s3://kafkasgdbucket1-
2/topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro
tes.avro
download: s3://kafkasgdbucket1-
2/topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro to
./tes.avro
root@stlrx2540m1-108:~#
```

22. 要打印记录，请使用 avro-tools-1.11.0.1.jar （可在中找到 "Apache 归档"）。

```
root@stlrx2540m1-108:~# java -jar /usr/src/avro-tools-1.11.0.1.jar
tojson tes.avro
21/10/30 00:20:24 WARN util.NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
{"f1":"value1"}
{"f1":"value2"}
{"f1":"value3"}
root@stlrx2540m1-108:~#
```

"接下来：融合自重新平衡集群。"

融合的自平衡集群

"先前版本： Kafka S3 连接器。"

如果您以前管理过 Kafka 集群，则可能会熟悉手动将分区重新分配给不同代理所带来的挑战，以确保在集群中平衡工作负载。对于部署了大型 Kafka 的组织来说，重新配置大量数据可能会令人望而生畏，繁琐且存在风险，尤其是在集群上构建任务关键型应用程序时。但是，即使对于最小型的 Kafka 使用情形，该过程也非常耗时，并且容易出现人为错误。

在我们的实验室中，我们测试了 Confluent 自平衡集群功能，该功能可根据集群拓扑变化或负载不平衡自动重新平衡。当节点故障或扩展节点需要在代理之间重新平衡数据时， Confluent 重新平衡测试有助于测量添加新代理的时间。在典型的 Kafka 配置中，要重新平衡的数据量会随着集群的增长而增加，但在分层存储中，重新平衡仅限于少量数据。根据我们的验证，在典型的 Kafka 架构中，分层存储的重新平衡需要几秒或几分钟的时间，并且随着集群的增长而线性增长。

在自平衡集群中，分区重新平衡完全自动化，可优化 Kafka 的吞吐量，加快代理扩展速度并减少运行大型集群的操作负担。在稳定状态下，自平衡集群会监控代理之间的数据偏差，并持续重新分配分区以优化集群性能。在纵向或横向扩展平台时，自平衡集群会自动识别是否存在新代理或删除旧代理，并触发后续分区重新分配。这样，您就可以轻松添加和停用代理，从而从根本上提高 Kafka 集群的弹性。这些优势不需要任何人工干预，复杂的数学运算，也不需要分区重新分配通常会带来人为错误的风险。因此，完成数据重新平衡所需的时间远远少于完成时间，您可以自由地专注于价值更高的事件流式项目，而无需持续监控集群。

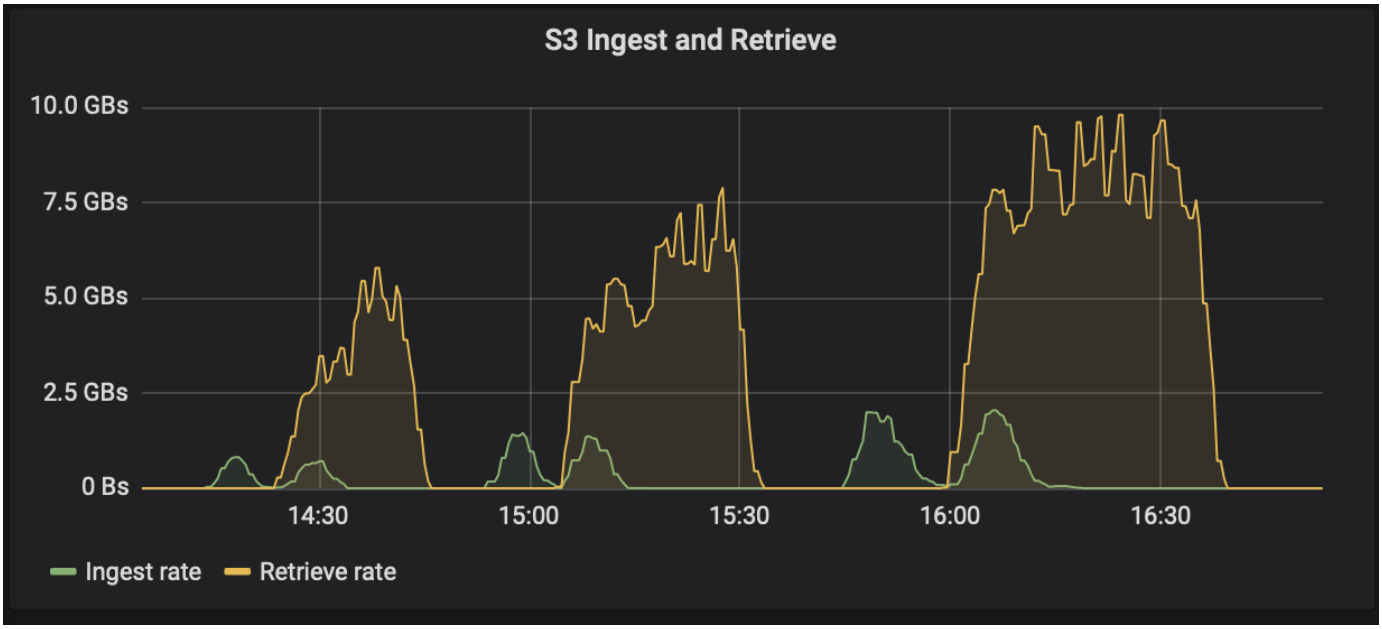
"接下来：最佳实践准则。"

最佳实践准则

"上一步：融合自重新平衡集群。"

- 根据我们的验证， S3 对象存储对于 Confluent 来说是保留数据的最佳选择。
- 我们可以使用高吞吐量 SAN （尤其是 FC ）来保留代理热数据或本地磁盘，因为在 Confluent 分层存储配置中， 代理数据目录中保留的数据大小取决于数据移动到对象存储时的区块大小和保留时间。
- 当 seg.bytes 较高时，对象存储可提供更好的性能；我们测试了 512 MB 。
- 在 Kafka 中，为主题生成的每个记录的密钥或值的长度（以字节为单位）由 `lage.key.value` 参数控制。对于 StorageGRID ， S3 对象载入和检索性能已提高到更高值。例如， 512 字节提供了 5.8 GBps 检索，1024 字节提供了 7.5 GBps S3 检索， 2048 字节提供了接近 10 Gbps 的值。

下图显示了基于 `length` 。 `key.value` 的 S3 对象载入和检索。



- * Kafka 调整。 * 要提高分层存储的性能，您可以增加 TierFetcherNumThreads 和 TierArchiverNumThreads 。一般情况下，您需要增加 TierFetcherNumThreads 以匹配物理 CPU 核数，并将 TierArchiverNumThreads 增加到 CPU 核数的一半。例如，在服务器属性中，如果您的计算机具有八个物理核心，请将 confuent.tier.fetcher.num.threads = 8 ，而将 confuent.tier.archiver.num.threads = 4 。
- * 主题删除的时间间隔。 * 删除主题后，不会立即开始删除对象存储中的日志段文件。而是在删除这些文件之前，有一个默认值为 3 小时的时间间隔。您可以修改配置 confluent.tier.topic.delete.check.interval.ms 以更改此间隔的值。如果删除某个主题或集群，也可以手动删除相应存储分段中的对象。
- * 分层存储内部主题上的 ACL 。 * 建议内部部署的最佳实践是，在分层存储使用的内部主题上启用 ACL 授权者。设置 ACL 规则，以便仅允许代理用户访问此数据。这样可以保护内部主题的安全，并防止对分层存储数据和元数据进行未经授权的访问。

```
kafka-acls --bootstrap-server localhost:9092 --command-config adminclient-
configs.conf \
--add --allow-principal User:<kafka> --operation All --topic "_confluent-
tier-state"
```

> 将用户 ` <Kafka>` 替换为部署中的实际代理主体。

例如，命令 `confuent-tier-state` 会在内部主题上为分层存储设置 ACL 。目前，只有一个内部主题与分层存储相关。此示例将创建一个 ACL ，为内部主题上的所有操作提供主体 Kafka 权限。

"接下来：规模估算。"

规模估算

"先前版本：最佳实践准则。"

Kafka 规模估算可通过四种配置模式来执行：简单，精细，反向和分区。

简单

简单模式适用于首次使用 Apache Kafka 的用户或早期使用情形。对于此模式，您可以提供吞吐量 MBps ，读取扇出，保留和资源利用率百分比等要求（默认为 60% ）。您还可以进入内部环境（裸机， VMware ， Kubernetes 或 OpenStack ）或云。根据这些信息， Kafka 集群的规模估算可提供代理， zookeeper ， Apache Kafka Connect 工作人员，架构注册表， REST 代理， ksqlDB 和 Confluent 控制中心所需的服务器数量。

对于分层存储，请考虑采用粒度配置模式来估算 Kafka 集群的规模。粒度模式适用于经验丰富的 Apache Kafka 用户或定义明确的用例。本节介绍了针对生产者，流处理器和使用者的规模估算。

生产者

要描述 Apache Kafka 的生成方（例如原生客户端， REST 代理或 Kafka 连接器），请提供以下信息：

- * 名称 * Spark 。
- * 生成方类型。 * 应用程序或服务，代理（ REST ， MQT ，其他）和现有数据库（ RDBMS ， NoSQL ，其他）。您也可以选择 " 我不知道 " 。
- * 平均吞吐量 * ，以每秒事件数（例如 1 ， 000 ， 000 ）表示。
- * 峰值吞吐量 * ，以每秒事件数（例如 4 ， 000 ， 000 ）表示。
- * 平均消息大小 * ，以字节为单位，未压缩（最大 1 MB ；例如 1000 ）。
- * 消息格式。 * 选项包括 Avro ， JSON ，协议缓冲区，二进制文件，文本， " 我不知道 " 等。
- * 复制因子。 * 选项包括 1 ， 2 ， 3 （ Confluent 建议）， 4 ， 5 ， 或 6.
- * 保留时间 * 一天（例如）。您希望将数据存储在 Apache Kafka 中多长时间？输入 -1 并输入任意单位，持续时间不受限制。计算器假定保留时间为 10 年，以实现无限保留。
- 选中 " 启用分层存储以减少代理数量并允许无限存储？ " 复选框
- 启用分层存储后，保留字段将控制存储在代理本地的热数据集。归档保留字段用于控制数据在归档对象存储中的存储时间。
- * 归档存储保留。 * 一年（例如）。您希望将数据存储在归档存储中多长时间？输入 -1 并输入任意单位，持续时间不受限制。计算器假定保留 10 年以实现无限保留。
- * 增长乘数 * 1 （例如）。如果此参数的值基于当前吞吐量，请将其设置为 1 。要根据其他增长调整大小，请将此参数设置为增长乘数。
- * 生产商实例数。 * 10 （例如）。将运行多少个生产商实例？要将 CPU 负载纳入规模估算计算，需要使用此输入。如果值为空，则表示 CPU 负载未计入计算中。

根据此示例输入，规模估算对生成方具有以下影响：

- 未压缩字节的平均吞吐量： 1 Gbps 。未压缩字节的峰值吞吐量： 4 Gbps 。以压缩字节为单位的平均吞吐量： 400 Mbps 。以压缩字节为单位的峰值吞吐量： 1.6 GBps 。此值基于默认 60% 的压缩率（您可以更改此值）。
    - 所需的总代理热设置存储： 31 ， 104 TB ，包括复制，已压缩。所需的代理外归档存储总量： 378 ， 432 TB ，已压缩。使用 … "https://fusion.netapp.com" 用于 StorageGRID 规模估算。

流处理器必须描述其应用程序或服务，这些应用程序或服务会使用 Apache Kafka 中的数据并生成回 Apache Kafka 。大多数情况下，这些数据存储库内置在 ksqlDB 或 Kafka 流中。

- * 名称 * 。 * Spark 流式传输器。

- * 处理时间。 * 此处理器处理单条消息需要多长时间？
  - 1 毫秒（简单，无状态转换） [示例] ， 10 毫秒（有状态内存操作）。
  - 100 毫秒（有状态网络或磁盘操作）， 1000 毫秒（第三方 REST 调用）。
  - 我已对该参数进行了基准测试，并确切了解它需要多长时间。
- * 输出保留 * 1 天（示例）。流处理器会将其输出返回到 Apache Kafka 。您希望将此输出数据存储在 Apache Kafka 中多长时间？输入 -1 并输入任意单位，持续时间不受限制。
- 选中 " 启用分层存储以减少代理数量并允许无限存储？ " 复选框
- * 归档存储保留。 * 1 年（例如）。您希望将数据存储在归档存储中多长时间？输入 -1 并输入任意单位，持续时间不受限制。计算器假定保留 10 年以实现无限保留。
- * 输出直通百分比 * 100 （例如）。流处理器会将其输出返回到 Apache Kafka 。多少百分比的入站吞吐量将输出回 Apache Kafka ？ 例如，如果入站吞吐量为 20Mbps ，而此值为 10 ，则输出吞吐量将为 2Mbps 。
- 从哪些应用程序读取此数据？选择 "Spark " ，即在基于生产商类型的规模估算中使用的名称。根据上述输入，估算流处理程序实例和主题分区估计值时可能会产生以下影响：
- 此流处理器应用程序需要以下数量的实例。传入的主题可能也需要这么多分区。请联系 Confluent 以确认此参数。
  - 1 ， 000 表示平均吞吐量，无增长乘数
  - 4 ， 000 表示峰值吞吐量，无增长乘数
  - 1 ， 000 表示使用增长乘数的平均吞吐量
  - 4 ， 000 表示峰值吞吐量，并使用增长乘数

使用者

请描述您使用 Apache Kafka 中的数据而不生成回 Apache Kafka 的应用程序或服务，例如原生客户端或 Kafka 连接器。

- * 名称 * 。 * Spark 使用者。
- * 处理时间。 * 此使用者需要多长时间处理一条消息？
  - 1 毫秒（例如，日志记录等简单无状态任务）
  - 10 毫秒（快速写入数据存储库）
  - 100 毫秒（写入数据存储库的速度较慢）
  - 1000 毫秒（第三方 REST 调用）
  - 已知持续时间的其他一些基准流程。
- * 使用者类型。 * 应用程序，代理或接收到现有数据存储库（ RDBMS ， NoSQL ，其他）。
- 从哪些应用程序读取此数据？将此参数与先前确定的生成方和流规模估算相连接。

根据上述输入，您必须确定使用者实例的规模估算和主题分区估算。使用者应用程序需要以下数量的实例。

- 2 ， 000 表示平均吞吐量，无增长乘数
- 峰值吞吐量为 8 ， 000 ，无增长乘数
- 2 ， 000 表示平均吞吐量，包括增长乘数

- 8 ， 000 表示峰值吞吐量，包括增长乘数

传入主题可能也需要此数量的分区。请联系 Confluent 进行确认。

除了对生产者，流处理器和使用者的要求之外，您还必须满足以下附加要求：

- * 重建时间。 * 例如， 4 小时。如果 Apache Kafka 代理主机发生故障，其数据丢失，并且配置了新主机来更换发生故障的主机，则新主机必须自行重建多快？如果此值未知，请将此参数留空。
- * 资源利用率目标（百分比）。 * 例如 60 。您希望主机在平均吞吐量期间的利用率如何？除非您使用的是 Confluent 自平衡集群，否则 Confuent 建议使用 60% 的利用率，在这种情况下，利用率可能会更高。

描述您的环境

- * 您的集群将在什么环境中运行？ * Amazon Web Services ， Microsoft Azure ， Google 云平台，内部裸机，内部部署 VMware ， 内部使用 OpenStack 还是内部使用 Kubernates ？
- * 主机详细信息。 * 核心数： 48 （例如），网卡类型（ 10GbE ， 40GbE ， 16GbE ， 1GbE 或其他类型）。
- * 存储卷。 * 主机： 12 （例如）。每个主机支持多少个硬盘驱动器或 SSD ？ 因此，建议每个主机配置 12 个硬盘驱动器。
- * 存储容量 / 卷（以 GB 为单位）。 * 1000 （例如）。单个卷可以存储多少 GB 的存储？这两者建议使用 1 TB 磁盘。
- * 存储配置。 * 如何配置存储卷？Confuent 建议使用 RAID10 来利用所有 Confluent 功能。JBOD ， SAN ， RAID 1 ， RAID 0 ， RAID 5 ， 此外，还支持其他类型。
- * 单卷吞吐量（ MBps ）。 * 125 （例如）。单个存储卷的每秒读取或写入速度（以 MB/ 秒为单位）有多快？ Confuent 建议使用标准硬盘驱动器，这些驱动器的吞吐量通常为 125 MBps 。
- * 内存容量（ GB ） .* 64 （例如）。

确定环境变量后，选择调整集群大小。根据上述示例参数，我们确定了以下 Confluent Kafka 规模估算：

- * Apache Kafka.* 代理计数： 22 。您的集群受存储限制。请考虑启用分层存储以减少主机数量并允许无限存储。
- * Apache zookeeper 。 * 计数： 5 ； Apache Kafka Connect Worker ：计数： 2 ；架构注册表：计数： 2 ； REST 代理：计数： 2 ； ksqlDB ：计数： 2 ； Confluent Control Center ：计数： 1 。

对平台团队使用反向模式，而不考虑使用情形。使用分区模式计算单个主题所需的分区数。请参见 https://eventsizer.io 用于根据反向和分区模式进行规模估算。

"接下来：总结。"

结论

"上一步：规模估算。"

本文档提供了将 Confluent 分层存储与 NetApp 存储结合使用的最佳实践准则，其中包括验证测试，分层存储性能结果，调整， Confluent S3 连接器以及自平衡功能。考虑到 ILM 策略，具有多个验证性能测试的流畅性能以及行业标准 S3 API ， NetApp StorageGRID 对象存储是流畅分层存储的最佳选择。

从何处查找追加信息

要了解有关本文档中所述信息的更多信息，请查看以下文档和 / 或网站：

- 什么是 Apache Kafka

  "https://www.confluent.io/what-is-apache-kafka/"

- NetApp 产品文档

  "https://www.netapp.com/support-and-training/documentation/"

- S3-sink 参数详细信息

  "https://docs.confluent.io/kafka-connect-s3-sink/current/configuration_options.html#s3-configuration-options"

- Apache Kafka

  "https://en.wikipedia.org/wiki/Apache_Kafka"

- 在 Confluent 平台中提供无限存储

  "https://www.confluent.io/blog/infinite-kafka-storage-in-confluent-platform/"

- 融合分层存储—最佳实践和规模估算

  "https://docs.confluent.io/platform/current/kafka/tiered-storage.html#best-practices-and-recommendations"

- 适用于 Confluent Platform 的 Amazon S3 Sink Connector

  "https://docs.confluent.io/kafka-connect-s3-sink/current/overview.html"

- Kafka 规模估算

  "https://eventsizer.io"

- StorageGRID 规模估算

  "https://fusion.netapp.com/"

- Kafka 用例

  "https://kafka.apache.org/uses"

- 在融合平台 6.0 中实现自我平衡的 Kafka 集群

  "https://www.confluent.io/blog/self-balancing-kafka-clusters-in-confluent-platform-6-0/"

  "https://www.confluent.io/blog/confluent-platform-6-0-delivers-the-most-powerful-event-streaming-platform-to-date/"

版本历史记录

version	Date	文档版本历史记录
版本 1.0	2021 年 12 月	初始版本。

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.